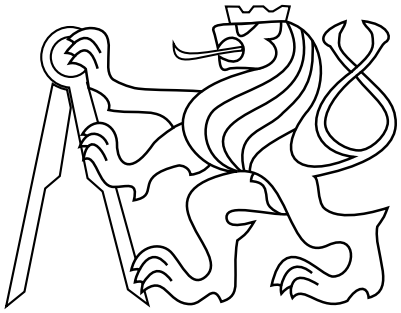
Czech Technical University in Prague

Faculty of Electrical Engineering
Department of Cybernetics
Multi-robot Systems



Improving Sim-to-Real transfer of Learning-based Policies for Quadcopter Racing through Domain Randomization

Semestral Project

Adam El Ghaib

Prague, January 2025

Study programme: Cybernetics and Robotics (Space Master)

Supervisor: Ing. Robert Pěnička, Ph.D.

Abstract

The study of autonomous Unmanned Aerial Vehicles (UAVs) has become a prominent sub-field of mobile robotics.

Keywords Unmanned Aerial Vehicles, Automatic Control

Abbreviations

UAV Unmanned Aerial Vehicle

RL Reinforcement Learning

DR Domain Randomization

POMDP Partially Observable Markov Decision Process

DoF Degrees of Freedom

Sim2Real Simulation-to-Seality

PPO Proximal Policy Optimization

AC Actor Critic

MLP Multi-Layer Perceptron

CTBR Collective Thrust and Body Rates

Contents

1 Introduction

In the context of drone racing, where quadcopters are pushed to their physical and computational limits, there is a growing trend toward using neural networks for control. As demonstrated in [1], this approach has made possible to outperform some of the world's most skilled human pilots.

These neural networks controllers are tipically optimized using RL algorithms. However, despite many of the advantadges that RL bring, it is still a hilghly sample inefficent algorithm that makes collecting data in the real world dangerous an expensive. Because of this, training is often performed in simulated environments that provide a safe and scalable alternative. Nonetheless, this alternative leads to other problems since the resulting policies, learned in simulations, often perform worse than expected or fail to transfer to real-world scenarios altogether. This discrepancy in the policy's performance between simulation and reality is known as the Simulation-to-Seality (Sim2Real) gap, and it mainly arises from an undesired overfit of the optimization process to the simulated scenario.

1.1 Related works

Demonstrating that Domain Randomization (DR) can significantly increase the adaptability and robustness of autonomous systems, the work of [2] serves as the direct inspiration for this project. Moreover in [3], the authors show how an appropiate system identification combined with DR effectively bridges the Sim2Real gap.

2 Methodology

2.1 Problem definition

We define a domain as a specific subset of possible Partially Observable Markov Decision Process (POMDP) environments. While a complete POMDP is formally characterized by an 8-tuple [4], we focus on a reduced 6-tuple representation that captures the essential elements for our domain definition. A standard domain is then characterized by $\mathcal{D} = (\mathcal{S}, \mathcal{A}, \mathcal{O}, P, O, R)$ where:

- $\mathcal{S}$: denotes a set of states called the state space.
- $\mathcal{A}$: is the set of actions called the action space.
- $\mathcal{O}$: is the set of observations called the observation space.
- $\mathbf{P}(\mathbf{s}'|\mathbf{s}, \mathbf{a})$: is the probability that a given action $a \in A$, at state $s \in S$ will lead to a new state $s' \subset S$.
- $\mathbf{O}(\mathbf{o}|\mathbf{s}', \mathbf{a})$: is the observation probability, representing the probability of observing $o \in \mathcal{O}$ after taking action a and transitioning to state s'.
- $\mathbf{R}(\mathbf{s}', \mathbf{s}, \mathbf{a})$: is the immediate reward, or expected inmediate reward, received for transitioning from s to s' after performing action a.

From the source domain, $\mathcal{D}_s$, (i.e. simulation), we can control a set of N parameters that define a configuration vector, $\boldsymbol{\xi} = [\xi_i, \xi_{i+1}, ..., \xi_N]$. These parameters will affect the transition probability $\mathbf{P}(\mathbf{s}'|\mathbf{s}, \mathbf{a})$ of the domain. Examples of ξ_i can include:

Physical properties	ξ_{mass}, ξ_{inertia}, ξ_{drag}, ξ_{gravity}
Perception properties	$\xi_{\text{sensor\_noise}}$, $\xi_{\text{sensor\_latency}}$
Viual properties	ξ_{lighting}, ξ_{texture}, ξ_{fov}
Scene properties	$\xi_{\text{gate\_pos}}$, $\xi_{\text{gate\_size}}$, , $\xi_{\text{gate\_num}}$

When DR is applied, each configuration $\boldsymbol{\xi}$ is sampled from a bounded randomization space Ξ with a probability distribution. That is to say, $\boldsymbol{\xi} \in \Xi \subset \mathbb{R}^N$, $\boldsymbol{\xi} \sim p(\boldsymbol{\xi})$ where $\xi_i \in [\xi_i^{\text{low}}, \xi_i^{\text{high}}]$. At the start of every training episode, a new environment insance is created by sampling a new configuration vector $\boldsymbol{\xi}$ from the distribution $p(\boldsymbol{\xi})$. Therefore, the agent will interact with a new POMDP environment, a new domain $\mathcal{D}_{\boldsymbol{\xi}}$. This way, the agent can collect data from a vareity of environments that otherwise would reamain unexplored.

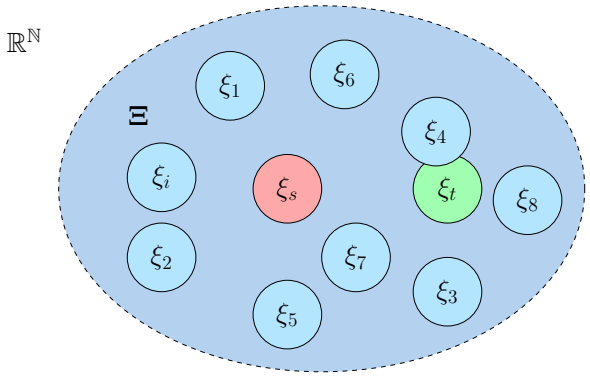


Figure 2.1: In red, the source domain randomization configuration. In light blue, ranodimized instance of the source configuration. In green, the target domain configuration.

Hereby, during training, the neural network weights and biases θ are optimized to maximize the expected reward R **across a distribution of randomized domains** [5]. With the aim of finding the optimal, θ^* that will lead to the optimal parametrized policy π^*_θ.

$$\theta^* = \arg\max_\theta \mathbb{E}_{\boldsymbol{\xi}\sim\Xi}\left[\mathbb{E}_{\pi_\theta,\tau\sim D_\xi}[R(\tau)]\right] \tag{2.1}$$

Where $R(\tau) = \sum_{t=0}^{n} \gamma^t r_t$ is the cummulative discounted reward over a trajectory (or sequence) of states and actions $\tau = (s_0, a_0, s_1, a_1, ...)$, for the episode length n.

As illustrated in Figure 2.1 and described in Equation 2.1, the policy is exposed to a variety of domains to promote generalization and improve robustness, increasing the likelihood of successful transfer to the real environment. However, this strategy further reduces the sample efficiency of RL and increases training time. Therefore, it is essential to properly bound the randomization space Ξ so that it encompasses only the minimum necessary range to include the real domain.

2.2 Quadcopter model

In the simulated environments, the quadcopter is modeeld as a 6 Degrees of Freedom (DoF), rigid body. The state of the quadcopter is defined by its position $\mathbf{p}$, velocity $\mathbf{v}$, orientation $\mathbf{q}$, body rates $\boldsymbol{\omega}$ and rotor speeds $\boldsymbol{\Omega}$. Namely, $\mathbf{x} = [\mathbf{p}, \mathbf{v}, \mathbf{q}, \boldsymbol{\omega}, \boldsymbol{\Omega}]^T$. The position, velocity and acceleration are respresented in the world frame $\mathcal{W} \in \mathbb{R}^3$, while the body rates in the body frame $\mathcal{B} \in \mathbb{R}^3$. The orientation is described by an unit quaternion rotation $\mathbf{q} \in \mathbb{SO}(3)$. Hence, the kinematics and dynamics of the drone are:

$$\dot{\mathbf{p}}^{\mathcal{W}} = \mathbf{v}^{\mathcal{W}} \tag{2.2}$$

$$\dot{\mathbf{q}} = \tfrac{1}{2}\,\mathbf{q} \odot \begin{bmatrix} 0 & \boldsymbol{\omega}^{\mathcal{B}} \end{bmatrix}^T \tag{2.3}$$

$$\dot{\mathbf{v}}^{\mathcal{W}} = \frac{R(\mathbf{q})}{m}\left(\mathbf{f}_T^{\mathcal{B}} + \mathbf{f}_D^{\mathcal{B}}\right) + \mathbf{g}^{\mathcal{W}} \tag{2.4}$$

$$\dot{\boldsymbol{\omega}}^{\mathcal{B}} = J^{-1}\left(\boldsymbol{\tau}^{\mathcal{B}} - \boldsymbol{\omega}^{\mathcal{B}} \times J\boldsymbol{\omega}^{\mathcal{B}}\right) \tag{2.5}$$

$$\dot{\boldsymbol{\Omega}} = \frac{1}{k_{\mathrm{m}}}\left(\boldsymbol{\Omega}_c - \boldsymbol{\Omega}\right) \tag{2.6}$$

Where, $\odot$ represents quaternion multiplication, $R(\mathbf{q})$ is the rotational matrix from body frame $\mathcal{B}$ to world frame $\mathcal{W}$ at orientation $\mathbf{q}$, J is the diagonal inertia matrix, $\mathbf{f}_T^{\mathcal{B}}$ and $\mathbf{f}_D^{\mathcal{B}}$ are the collective thrust and drag respectively, k_m the motor time constant, m the qudcopter's mass and $\mathbf{g}$ is gravity.

In the body frame $\mathcal{B}$, the individual motor thrusts $T_i,\ i = 1, .., 4$, are assumed to be perpendicular to the $\mathbf{xy}$ plane. The collective thrust $\mathbf{f}_T^{\mathcal{B}}$, and torques $\boldsymbol{\tau}^{\mathcal{B}}$, can be maped from the individual motor thrusts with the allocation matrix Γ.

$$\begin{bmatrix} \mathbf{f}_T^{\mathcal{B}} \\ \tau_x \\ \tau_y \\ \tau_z \end{bmatrix} = \begin{bmatrix} 1 & 1 & 1 & 1 \\ d/\sqrt{2} & -d/\sqrt{2} & -d/\sqrt{2} & d/\sqrt{2} \\ -d/\sqrt{2} & -d/\sqrt{2} & d/\sqrt{2} & d/\sqrt{2} \\ c_{tf} & -c_{tf} & c_{tf} & -c_{tf} \end{bmatrix} \begin{bmatrix} T_1 \\ T_2 \\ T_3 \\ T_4 \end{bmatrix} \tag{2.7}$$

Being d the arm length and c_{tf} the linear torque constant. Likewise, the indidual motror thrusts T_i, can be mapped from the rotor speeds Ω_i with the thrust map coefficents, $\mathbf{k_T} = [a_0,\ a_1,\ a_2]$ that are determined experimentally fitting a quadratic curve.

$$T_i = \mathbf{k_T} \begin{bmatrix} \Omega_i^2 \\ \Omega_i \\ 1 \end{bmatrix} = \begin{bmatrix} a_0 & a_1 & a_2 \end{bmatrix} \begin{bmatrix} \Omega_i^2 \\ \Omega_i \\ 1 \end{bmatrix} \tag{2.8}$$

The total drag force, $\mathbf{f}_D^{\mathcal{B}}$ is calculated using a simplified aerodynamic model function of the dron's linear velocity and proportional to the drag coefficents.

$$\mathbf{f}_D^{\mathcal{B}} = \begin{bmatrix} C_{D_x} & 0 & 0 \\ 0 & C_{D_y} & 0 \\ 0 & 0 & C_{D_z} \end{bmatrix} \begin{bmatrix} v_x^{\mathcal{B}} \\ v_y^{\mathcal{B}} \\ v_z^{\mathcal{B}} \end{bmatrix} \tag{2.9}$$

2.3 RL Set Up

Although the simplified quadcopter model defined in 2.2 is used to represent the drone dynamics in simulation, the RL setup employed in this project is **model-free**. Specifically, it consists of an Actor Critic (AC) structure optimized using Proximal Policy Optimization (PPO) [6]. The implementation relies on Stable-Baselines3 [7] to interface with a custom C++ Gymnasium environment (based on flightmare [8]). To train the AC networks PPO is used.

The actor policy netwrok π_θ, is defined by a Multi-Layer Perceptron (MLP) that takes an observation vector at time step t, of 31 inputs (see Equation 2.10). The observation vector is sampled from a bounded observation space $o_t \in \mathcal{O} \subset \mathbb{R}^{31}$.

$$o_t = \begin{bmatrix} \mathbf{p}_t & \mathbf{v}_t & \text{vec}R(\mathbf{q})_t & u_{t-1} & \mathbf{p}_{\mathbf{g}_t}^{TL} & \mathbf{p}_{\mathbf{g}_t}^{TR} & \mathbf{p}_{\mathbf{g}_t}^{BL} & \mathbf{p}_{\mathbf{g}_t}^{BL} & \mathbf{p}_t^{\text{look}} \end{bmatrix}^T \tag{2.10}$$

Where u_{t-1} is the throttle motor command at the prevous time step. $\mathbf{p}_{\mathbf{g}_t}^X$ are relative positions of the corners of the target gate at time t with respect the body frame $\mathcal{B}$. And, $\mathbf{p}_t^{\text{look}}$ is the look at position of the target gate at time t, also in the body frame. The input layer is followed by three hidden layers of 128 neurons each, and it outputs 4 neurons. The outputs are the Collective Thrust and Body Rates (CTBR), sampled from a bounded action space $a_t \in \mathcal{A} \subset \mathbb{R}^4$.

$$a_t = \begin{bmatrix} \mathbf{f}_{Tt} & \boldsymbol{\omega}_t \end{bmatrix}^T \tag{2.11}$$

On the other hand, the critic network aritecthure V_f, is formed also by 3 hidden layers but of 256 neurons each, it has the same input space and it oputs a single value based on the observation o_t.

Reward System

- mention also training steps, time and so on

2.4 DR configuration

The randomiztion configuration condsidered for this semestral project has been the following:

$$\boldsymbol{\xi}_d = [m, J, C_D, k_m, \mathbf{k_T}, \Gamma]$$

Every parameter is relatively randomized (multiplicative randomization) by a constant sampled from an uniform distribution $\mathcal{U}(1-p, 1+p)$ parametrized by the parameter p which indicates the randomization percentage.

3 Results

To investigate how DR affects the transferability of controllers learned in simulation to different environments, the following experiments were designed. First, three randomized drone instances were initialized in simulation, and a common policy was executed concurrently in Gazebo during 5 minutes to measure the uav's lap times and success rates. The policies were trained to race in a simple eight-figure track (see Figure 3.1) and their dynamical parameters can be found in Table A.1. This procedure was performed for four distinct policies trained at randomization levels of 0%, 10%, 20%, and 30%. The results of this first experiment are presented in section 3.1. Following successful validation in the simulated environment, the experiments were replicated on physical hardware with.....

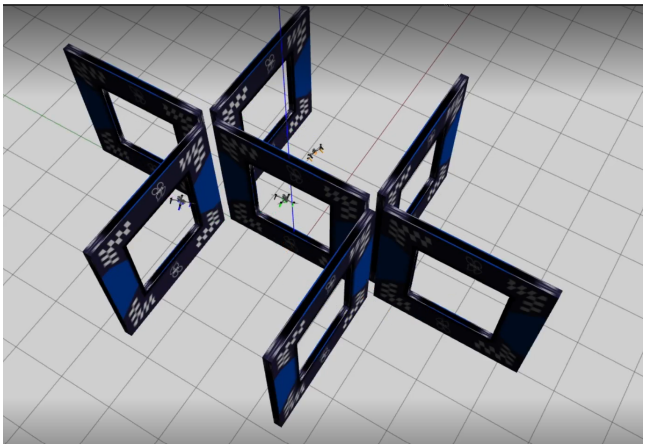


Figure 3.1: Three randomized drones racing in Gazebo.

3.1 Simulation to Simulation Results

As depicted in Figure 3.2, the baseline configuration with 0% randimization yields to the fastest avarage lap time of 4.13s. However, this speed comes with a significat cost in realaiability. More than half of the laps resulted in gate crash and two of the drones fatally crashed druing the experiment.

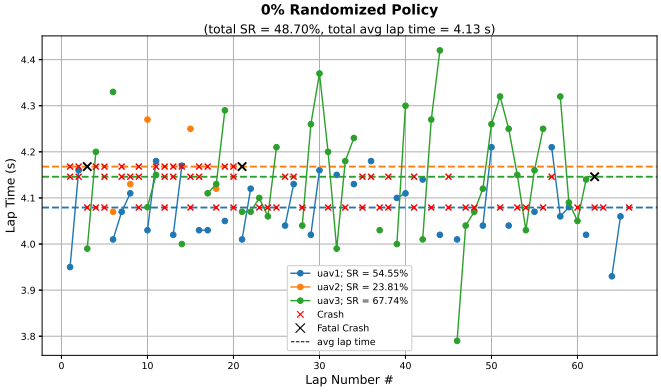


Figure 3.2: 0% randomized policy simulated results.

Randomizing the dynamical parameters at 10% produced a notable increase in average lap times to 5.79s, Figure 3.3. Nonetheless, reliability and robustness improved significantly, with all three drones completing the experiment without fatal crashes and success rates exceeding 97%.

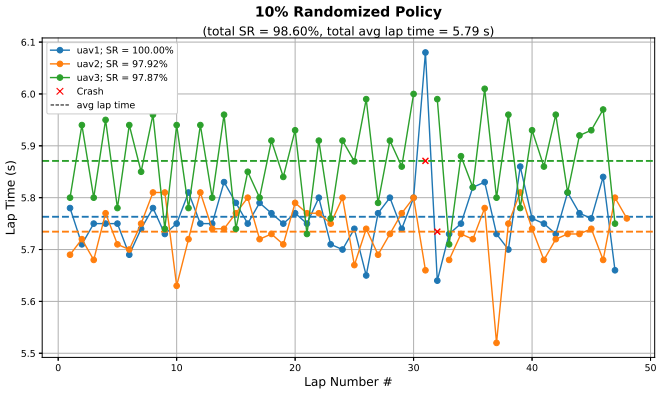


Figure 3.3: 10% randomized policy simulated results.

A further increase in randomization to 20% resulted in slightly reduced lap times. It is worth noting that training is a stochastic process; training two policies with identical initialization parameters yield to different results. Among all trained policies, the 20% randomized policy achieved the best overall performance with an average lap time of 5.38s and a perfect success rate across all three drones, as shown in Figure 3.4.

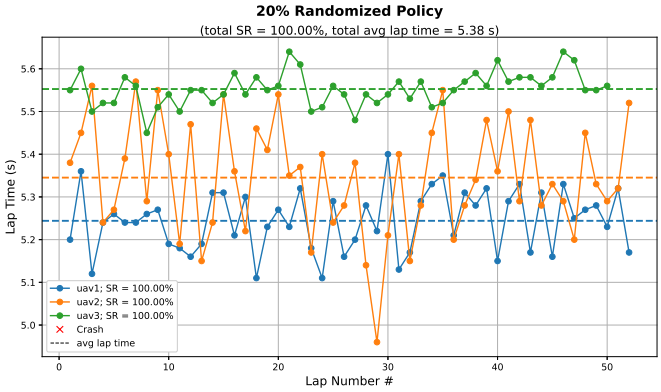


Figure 3.4: 20% randomized policy simulated results.

Finally, training with 30% randomization yielded acceptable results for UAV1 and UAV2, as shown in Figure 3.5. However, for UAV3, the policy failed to reliably control the drone. As discussed in the Methodology section, we hypothesize that excessive randomization can destabilize training, worsening sample efficiency and requiring more convergence time.

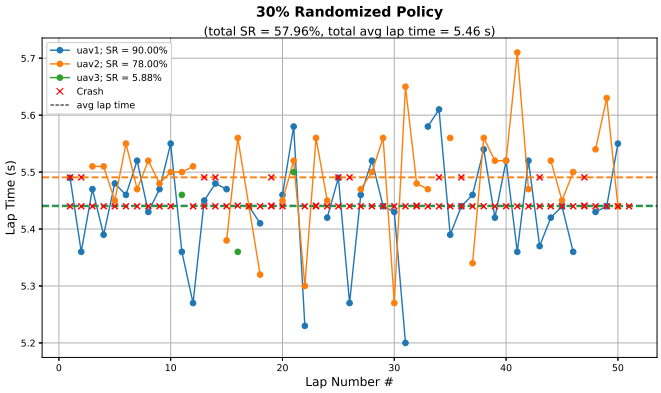


Figure 3.5: 30% randomized policy simulated results.

Additionally, the selected randomization methodology randomizes all dynamical parameters uniformly, but some critical parameters such as the thrust map may require more restrictive bounds to prevent physically infeasible configurations.

3.2 Simulation to Reality Results

Policy	UAV	**Success Rate**[%]		**Number of Laps**#		**Avg Lap Time**[s]		**Fatal Crash**	
		Sim	Real	Sim	Real	Sim	Real	Sim	Real
0% DR	a300								
	uav1	54.55		36		4.08		No	
	uav2	23.81		5		4.17		**Yes**	
	uav3	67.74		42		4.15		**Yes**	
10% DR	a300								
	uav1	100		47		5.76		No	
	uav2	97.92		47		5.73		No	
	uav3	100		46		5.87		No	
20% DR	a300								
	uav1	100		52		5.24		No	
	uav2	100		52		5.35		No	
	uav3	100		50		5.55		No	
30% DR	a300								
	uav1	90		45		5.44		No	
	uav2	78		39		5.49		No	
	uav3	5.88		3		5.44		No	

Table 3.1: (sim time 5 min)

4 Conclusion

This work investigated the effectiveness of Domain Randomization (DR) in bridging the simulation-to-reality gap for learning-based quadcopter racing controllers. By systematically varying the randomization level of key drone dynamics parameters, we demonstrated that DR significantly improves policy robustness and transferability across different vehicle platforms.

Our simulated experiments revealed a clear trade-off between control performance and reliability. The baseline policy trained without randomization (0% DR) achieved the fastest lap times of 4.13s but suffered from poor stability, with over 50% of laps resulting in gate collisions and two fatal crashes among the three tested drone instances. In contrast, policies trained with moderate randomization (10% and 20% DR) exhibited substantially improved success rates exceeding 97%, while maintaining competitive lap times of 5.73s and 5.38s respectively. The 20% DR configuration emerged as the optimal balance, achieving a perfect 100% success rate across all three simulated drones without sacrificing significant performance.

However, our results also indicate diminishing returns and potential instability at higher randomization levels. The 30% DR policy showed degraded performance, particularly for UAV3, suggesting that excessive parameter randomization can destabilize the learning process and reduce sample efficiency. This aligns with our hypothesis that the randomization space must be carefully bounded to encompass the real domain characteristics without introducing unnecessary variability that impedes convergence.

5 References

[1] Elia Kaufmann et al. "Champion-level drone racing using deep reinforcement learning". In: *Nature* 620.7976 (Aug. 2023), 982–987. ISSN: 1476-4687. DOI: 10.1038/s41586-023-06419-4. URL: http://dx.doi.org/10.1038/s41586-023-06419-4.

[2] Robin Ferede et al. "One Net to Rule Them All: Domain Randomization in Quadcopter Racing Across Different Platforms". In: (2025). DOI: 10.48550/ARXIV.2504.21586. URL: https://arxiv.org/abs/2504.21586.

[3] Jiayu Chen et al. "What Matters in Learning A Zero-Shot Sim-to-Real RL Policy for Quadrotor Control? A Comprehensive Study". In: (2024). DOI: 10.48550/ARXIV.2412.11764. URL: https://arxiv.org/abs/2412.11764.

[4] K.J Åström. "Optimal control of Markov processes with incomplete state information". In: *Journal of Mathematical Analysis and Applications* 10.1 (1965), pp. 174–205. ISSN: 0022-247X. DOI: https://doi.org/10.1016/0022-247X6590154-X. URL: https://www.sciencedirect.com/science/article/pii/0022247X6590154X.

[5] Lilian Weng. "Domain Randomization for Sim2Real Transfer". In: *lilianweng/github/io* (2019). URL: https://lilianweng.github.io/posts/2019-05-05-domain-randomization/.

[6] John Schulman et al. *Proximal Policy Optimization Algorithms*. 2017. DOI: 10.48550/ARXIV.1707.06347. URL: https://arxiv.org/abs/1707.06347.

[7] Antonin Raffin et al. "Stable-Baselines3: Reliable Reinforcement Learning Implementations". In: *Journal of Machine Learning Research* 22.268 (2021), pp. 1–8. URL: http://jmlr.org/papers/v22/20-1364.html.

[8] Yunlong Song et al. *Flightmare: A Flexible Quadrotor Simulator*. 2020. DOI: 10.48550/ARXIV.2009.00563. URL: https://arxiv.org/abs/2009.00563.

A Appendix A

Next table presents the parameter multipliers used to randomize the three simulated drones in Gazebo. The parameter p is sampled from an uniform distribution.

Parameter	**Nominal**	**Randomized** Parameter $* \mathcal{U}(1-p, 1+p)$		
		uav1 p = 10%	**uav2 p=20%**	**uav3 p=30%**
mass [kg]	1.178	1.041	0.814	1.212
body_radius [m]	0.05	0.964	0.883	0.702
body_height [m]	-0.1	1.023	1.177	1.271
mass_prop [kg]	0.008	1.095	1.143	1.191
radius_prop [m]	0.089408	1.072	0.918	1.288
motor_mesh_z_offset [m]	-0.0086	0.924	0.824	0.924
rotor_xy_offset [m]	0.1	1.067	1.192	1.067
rotor_z_offset [m]	-0.055	0.912	1.118	0.912
rotor_velocity_slowdown_sim	0.0159236	1.034	1.132	0.713
motor_constant	20.21	1.081	0.804	1.229
moment_constant	0.012	0.922	1.162	0.731
time_constant_up [s]	1/(20.5761)	0.953	0.886	1.221
time_constant_down [s]	1/(18.3486)	1.067	1.189	0.708
max_rot_velocity	1.0	1.058	0.833	1.197
rotor_drag_coefficient	0.1	1.041	1.144	1.276
inertia_body_radius [m]	0.25	1.091	0.817	1.298
inertia_body_height [m]	0.05	0.912	1.178	0.744

Table A.1: Parameter multipliers for 10%, 20%, and 30% randomized UAVs. Nominal column retains original values.